

学 号:	2025303025
------	------------

武汉理工大学

课程名称 企业级应用软件设计与开发

项目名称 智能旅游助手

方 向 Agentic AI 原生开发

学 号 2025303025

姓 名 曹熙佳

专 业 软件工程

指导老师 戚欣

提交日期 2026 年 6 月 22 日

目录

- 一、选题背景与设计思想4
 - 1.1 问题定义 4
 - 1.2 现有方案不足4
 - 1.3 项目价值 5
 - 1.4 设计思想与技术路线 5
- 二、Specs 规格文档7
 - 2.1 Product Spec7
 - 2.2 Architecture Spec7
 - 2.3 API Spec8
- 三、系统架构与设计 9
 - 3.1 核心架构图 9
 - 3.2 Agent 交互流程 11
 - 3.3 数据流设计13
- 四、关键实现与代码展示 14
 - 4.1 Agent 核心循环 14
 - 4.2 子 Agent 包装为工具 14
 - 4.3 MCP 客户端与缓存 15
 - 4.4 配置文件与上游兼容修复15
 - 4.5 Web 端实现16
- 五、测试与评估 16
 - 5.1 功能测试 16

5.2 Agent 行为评估	17
5.3 Benchmark 设计	17
5.4 Demo 截图	18
六、系统升级与扩展	19
6.1 可扩展架构	19
6.2 下一阶段计划	20
6.3 AI 能力演进路径	20
七、课程总结	20
7.1 个人收获	20
7.2 工程思维转变	21
7.3 对课程的建议	21

一 选题背景与设计思想

1.1 问题定义

旅行规划看起来是一个普通的信息检索问题，但真正做起来会发现它其实是一个连续决策问题。用户不会只问“北京有什么景点”，而是会把时间、预算、住宿位置、交通方式、兴趣偏好、同行情况等条件放在一起，希望系统直接给出一套能落地执行的安排。例如项目中的示例需求是“北京 3 日游，喜欢自然风光和历史文化，中等预算”，这句话里同时包含目的地、行程长度、兴趣约束、预算约束。系统如果只返回景点清单，实际上并没有解决用户的问题；它还需要考虑每天去哪里、景点之间的路线、天气是否影响户外活动、酒店是否方便、餐饮和交通大概花多少钱。

因此，本项目把“旅行规划”定义为一个由大模型负责理解与编排、由外部工具负责事实查询的 Agent 任务。用户输入自然语言后，系统需要自动拆解任务，分别查询天气、搜索景点、推荐酒店、规划路线，并把这些结果重新组织成按天排列的行程。最终输出不仅要有文字描述，还要有结构化 JSON，便于 Web 页面渲染、预算汇总和 Markdown 下载。

1.2 现有方案不足

现有的旅行规划方式最大的问题不是“查不到信息”，而是信息太分散、太难整合。用户规划一次旅行时，通常要先去天气软件看目的地天气，再去地图软件查景点位置和路线，又要到酒店平台比较住宿位置 and 价格，还会翻攻略社区了解哪些景点值得去、哪些餐厅适合顺路安排。每个平台都能解决一小块问题，但没有一个统一视图能把这些信息自然地串成一条完整行程。用户最后往往是在多个网页和应用之间来回切换，手动复制地址、估算距离、调整顺序，规划过程很容易变成一项繁琐的整理工作。

如果直接使用普通大模型来生成行程，体验会轻松一些，但它又会带来另一个问题：模型本身并不真正知道实时天气、最新 POI 状态、具体地址和路线耗时。它可能会给出听起来很合理的建议，却把已经变化的信息当成事实，甚至编造并不存在的景点细节。对于旅行这种强依赖现实位置和时间信息任务来说，这类错误会直接影响用户决策。传统攻略内容也存在类似局限，它们大多是静态文章，适合参考，但很难根据用户的具体日期、预算、住宿区域和兴趣点及时重组。也就是说，现有方

案要么让用户自己承担大量整合成本，要么给出不够实时、不够可靠的自动化建议，距离“一站式、可执行、个性化”的智能旅行规划还有明显差距。

1.3 项目价值

本项目的价值在于把大模型从“给建议的人”变成“会调用工具完成任务的规划助手”。在系统中，大模型不再单纯依赖已有知识回答，而是通过 MCP 协议接入高德地图工具，实时查询天气、POI 和路线信息。这样做可以减少凭空生成的风险，也能让最终行程更贴近真实场景。用户看到的不只是几段泛泛的旅游建议，而是一份包含天气、酒店、景点、餐饮、交通和预算的完整计划。

从工程角度看，这个项目也比较适合作为课程设计，因为它不是单一页面或单一接口的堆叠，而是包含了 Agent 架构、工具调用、流式输出、配置管理、异常诊断、缓存设计和前端展示的完整链路。它能体现 Agentic AI 原生开发的特点：模型负责理解和决策，工具负责获取外部事实，代码负责把两者组织成稳定产品。

1.4 设计思想与技术路线

在系统架构设计上，本项目摒弃了“将所有能力和指令塞进单一庞大提示词”的传统做法，转而采用“总控 Agent + 领域专家 Agent + MCP 工具”的分层多智能体（Multi-Agent）架构，并深度融合了多种经典设计模式以保证系统的高内聚与低耦合。

首先，在顶层调度控制上，总控 Agent TripPlanner 完美应用了门面模式（Facade）。它对外仅暴露极简的 `invoke()` 与 `stream()` 接口，将内部多 Agent 协同、任务拆解与状态流转的复杂性彻底隐藏。

其次，在业务执行层，天气、景点、酒店等具体的规划任务被精准分发给对应的 Specialist Agent（领域专家）处理。为了规范专家的构建并保留扩展性，代码通过 `SpecialistAgent.build()` 引入了模板方法模式（Template Method），定义了核心骨架，允许不同的专家子类按需覆盖构建逻辑。同时，为了让总控 Agent 能够无缝调度这些复杂的专家，系统巧妙地使用了 `@tool` 包装子 Agent，利用装饰器模式（Decorator）将它们平滑适配为标准的 LangChain Tool 接口。此外，面对众多的工具集，系统通过 `tool_domains` 字典运用了策略模式（Strategy），将工具按领域严格分组，使得总控大脑可以在运行时根据具体的任务场景动态选择并加载相应的工具策略。

再者，在底层基础设施层，系统需要与高德地图等外部 MCP 服务进行频繁交互。为了保障性能与资源安全，`McpClientManager` 采用了单例模式（Singleton），通过 `__new__` 与 `__initialized` 双

重保护机制，确保全局始终只有一个唯一的 MCP 连接，避免了连接泄露与资源浪费。对于驱动各 Agent 的大语言模型，则统一交由 `Config.create_llm()` 这个工厂方法（Factory Method）来创建，彻底隔离了不同模型复杂的构造与初始化细节。

最后，在视图展现侧，`render.py` 扮演了适配器模式（Adapter）的角色，将 Agent 输出的标准 JSON 规划数据，灵活适配为 CLI 命令行文本和 Streamlit 交互式组件两种截然不同的视图展现格式。

这种融合了多种设计模式的分层架构带来了显著的优势：不仅职责边界极其清晰，确保每个 Agent 专注且高效地处理自己擅长的专业领域，更赋予了系统极强的可扩展性。后续如果需要增加餐饮推荐、机票预订、火车票查询等全新能力，开发者完全可以按照同样的模式横向平滑扩展，无需重构现有核心代码，真正实现了符合“开闭原则”的可持续演进。

在技术路线上采用了前沿的 Agent 架构与声明式开发范式，将整体系统划分为核心推理层、工具集成层、交互呈现层及基础环境层：

核心大模型与 Agent 框架：系统以强大的通义千问（qwen3-max）作为基础语言模型，通过阿里百炼平台的 DashScope API 提供核心的推理与文本生成能力，并借助 `langchain_community.ChatTongyi` 适配器完成模型的无缝接入。在执行逻辑上，深度应用了最新版的 LangChain 与 LangGraph 框架来构建 ReAct Agent。这不仅实现了 Agent 的快速创建与灵活的工具编排，更保障了复杂多步任务下的状态流转与逻辑闭环。

协议级工具扩展（MCP）：为了突破大语言模型的信息孤岛，系统在工具集成层面创新性地引入了 MCP（Model Context Protocol）协议。底层依托 `mcp (1.24.0)` 运行时提供基于 `streamable-http` 的基础通信机制，上层则使用 `langchain_mcp_adapters (0.2.2)` 作为客户端桥梁，稳定高效地连接了阿里百炼高德地图 MCP 服务，赋予了 Agent 实时、精准调用现实世界地理数据的能力。

双轨用户交互与声明式 UI：系统针对不同的使用场景提供了灵活的双端体验。极客与开发者侧可通过 `Agent.py` 直接在命令行（CLI）中进行底层逻辑调试与演示；而普通用户则可以通过运行 `app.py` 访问基于 Streamlit (1.32.0) 构建的现代化图形界面。该 Web 端采用了经典的“侧边栏 + 主区域”声明式布局，大幅降低了交互门槛并提升了信息密度。

精准的结构化输出与工程环境：在数据输出侧，系统摒弃了不可控的纯文本生成，通过 `prompts.py` 强约束模型输出符合预期规范的 JSON Schema。这些结构化数据随后被移交给

render.py 与 Streamlit Web 组件，进行最终的精细化解析与界面展示，确保信息呈现的清晰度与准确性。

现代化的底层支撑：整套架构构建于 Python 3.12+ 现代运行环境之上。系统全面启用了异步编程 (asyncio) 以应对网络 I/O 密集型任务，并使用了最新的类型注解标准 (PEP 604) 以提升代码的可读性与健壮性；同时配合 python-dotenv 进行优雅的环境变量与配置管理（如 API Key 隔离），保障了项目的工程化落地与安全性。

二、Specs 规格文档

2.1 Product Spec

产品目标是做一个轻量但完整的智能旅行规划助手。它面向的用户不是专业旅行社，而是希望快速安排个人或小团队行程的普通用户。用户只需要输入目的地、日期范围、交通偏好、住宿偏好、旅行兴趣和补充要求，系统就要生成一份可以直接参考的旅行计划。这个计划应当包括每天的主要景点、推荐酒店、三餐建议、天气情况、路线交通方式和费用估算。

规格项	具体内容
目标用户	希望快速生成国内城市旅行计划的普通游客、学生、小团队出行者。
核心输入	目的地城市、开始日期、结束日期、交通方式、住宿类型、旅行偏好、额外要求。
核心输出	按天组织的行程、天气信息、酒店信息、景点信息、餐饮建议、预算汇总和总体建议。
主要交互	Web 端通过侧边栏填写参数并点击开始规划；CLI 端通过预设输入演示流式输出。
非功能要求	输出结构稳定、工具调用过程可见、异常信息可诊断、结果可下载。

产品设计中比较重要的一点是，Web 页面不是简单显示模型原文，而是把结果拆成更适合阅读的模块。天气用卡片展示，每日行程用 Tab 切换，预算用指标卡汇总，原始结果可以保存为 Markdown。这让系统更像一个可用的小产品，而不是一次性的聊天记录。

2.2 Architecture Spec

系统架构分为四层：用户界面层、Agent 编排层、MCP 工具层和外部服务层。用户界面层负责接收输入和展示结果；Agent 编排层负责理解需求、拆解任务并调用工具；MCP 工具层负责把地图能力包装成 LangChain Tool；外部服务层则是阿里百炼托管的高德地图 MCP 服务。



TripPlanner 并不直接写死每一种底层 API 调用，而是通过 McpClientManager 获取工具列表，再把其中属于 poi、weather、route 的工具分配给不同模块。这让系统在结构上具有一定弹性：如果 MCP 服务新增工具，只需要在配置中更新领域映射，Agent 层就可以按需使用。

2.3 API Spec

本项目的 API Spec 主要描述系统内部模块之间的调用接口，而不是面向公网开放的 HTTP API。由于系统采用 Streamlit + Agent 本地应用形态，核心接口集中在规划器、领域专家 Agent、MCP 客户端和结果渲染器之间。API 设计的目标是明确模块边界，使 Web 层只负责收集输入和展示结果，Agent 层负责任务编排，MCP 层负责外部工具访问，渲染层负责结构化结果解析。

一次完整规划请求的调用链为：app.build_prompt() 将 Web 表单参数转换为自然语言需求，随后调用 TripPlanner.stream() 进入总控 Agent 流程。TripPlanner 在内部通过 SpecialistAgent.invoke() 调用酒店、景点和天气领域 Agent，并通过 McpClientManager.get_tools_for() 获取对应 MCP 工具。模型生成完成后，render.parse_plan() 从输出文本中提取 JSON，供 Streamlit 页面渲染。

接口设计中，TripPlanner.invoke(user_input) 用于非流式调用，适合测试、脚本验证或一次性生成完整结果；TripPlanner.stream(user_input) 用于流式调用，适合 Web 和 CLI 场景，可以实时返回 token 与工具调用状态。SpecialistAgent.invoke(user_input) 封装单领域 ReAct Agent，使总控 Agent 不需要直接处理具体工具细节。McpClientManager.get_tools_for(domain) 根据 poi、

weather、route 等领域返回工具子集，避免 Agent 持有过多无关工具。render.parse_plan(text) 负责从模型混合输出中提取 JSON，如果解析失败则返回 None，由上层决定展示原文或提示错误。

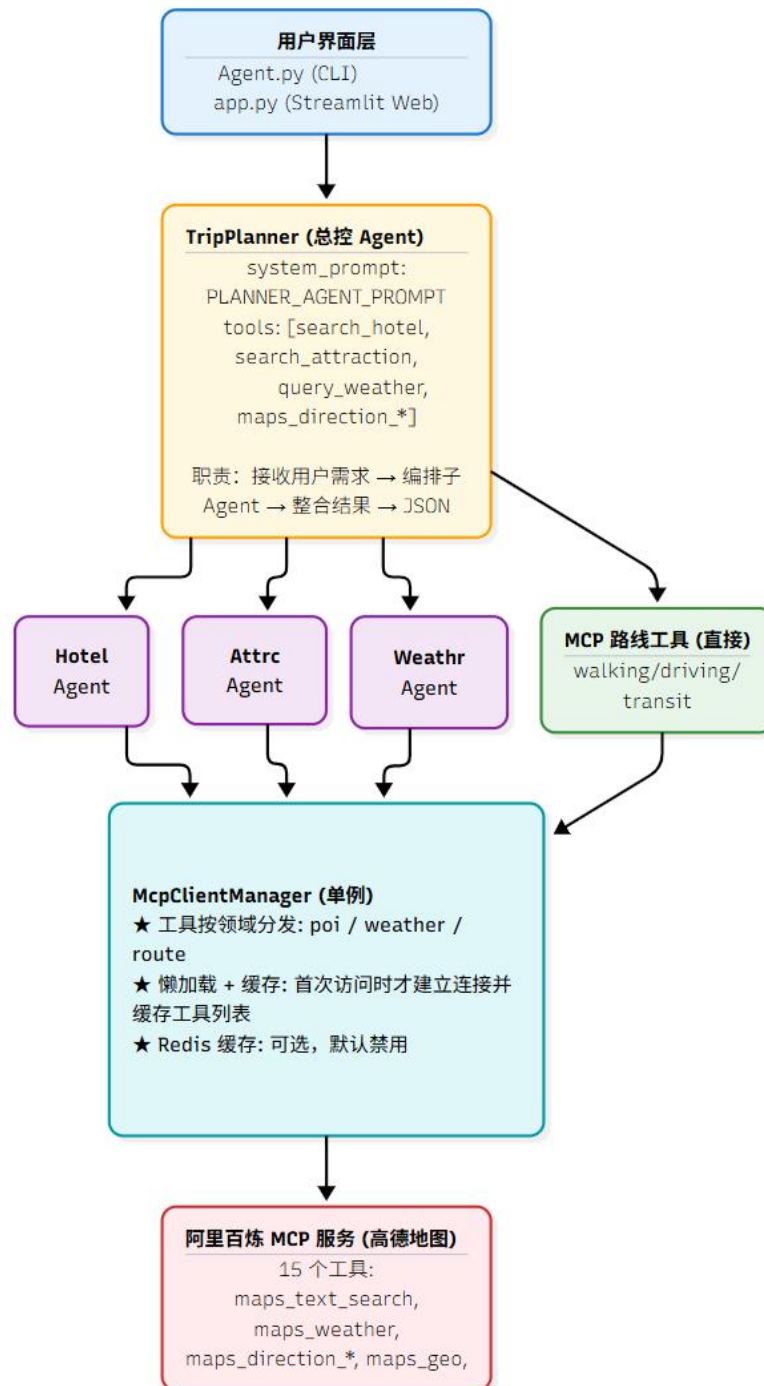
接口/函数	输入	输出	说明
TripPlanner.invoke(user_input)	自然语言旅行需求	完整结果文本	非流式调用，适合测试或一次性生成。
TripPlanner.stream(user_input)	自然语言旅行需求	AsyncIterator[str]	流式返回 token 和工具调用状态，适合 CLI/Web 展示。
SpecialistAgent.invoke(user_input)	领域查询语句	领域查询结果	封装单领域 ReAct Agent。
McpClientManager.get_tools_for(domain)	poi/weather/route	BaseTool 列表	按领域过滤 MCP 工具，减少误调用。
render.parse_plan(text)	混合文本	dict None	从模型输出中截取并解析 JSON。
app.build_prompt(...)	表单参数	自然语言 prompt	把 Web 结构化输入转为 Agent 能理解的描述。

这些接口共同体现了 SDD 中的“高内聚、低耦合”原则：Web 层不直接访问 MCP，Planner 不直接关心页面展示，SpecialistAgent 不负责全局行程组织，render.py 也不参与 Agent 推理。各模块通过清晰的输入输出连接，降低了修改某一层时对其他层的影响。

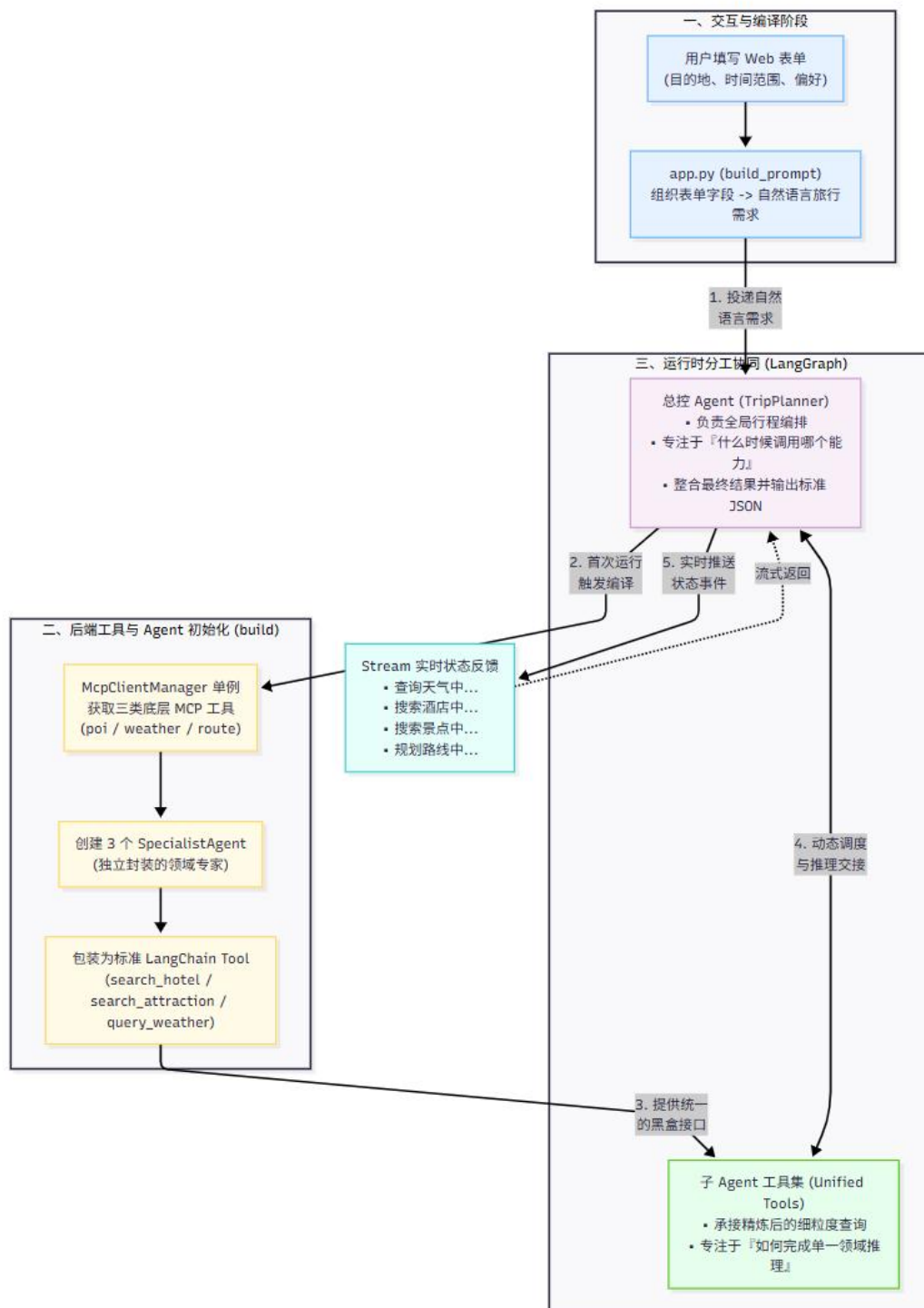
三、系统架构与设计

3.1 核心架构图

系统核心架构可以理解为一个从用户意图到外部工具、再到结构化结果的链路。用户输入不是直接交给工具，而是先交给总控 Agent，由它判断需要调用哪些子 Agent 和路线工具。子 Agent 得到任务后再调用受限的 MCP 工具，最后由 Planner 汇总所有结果。



3.2 Agent 交互流程

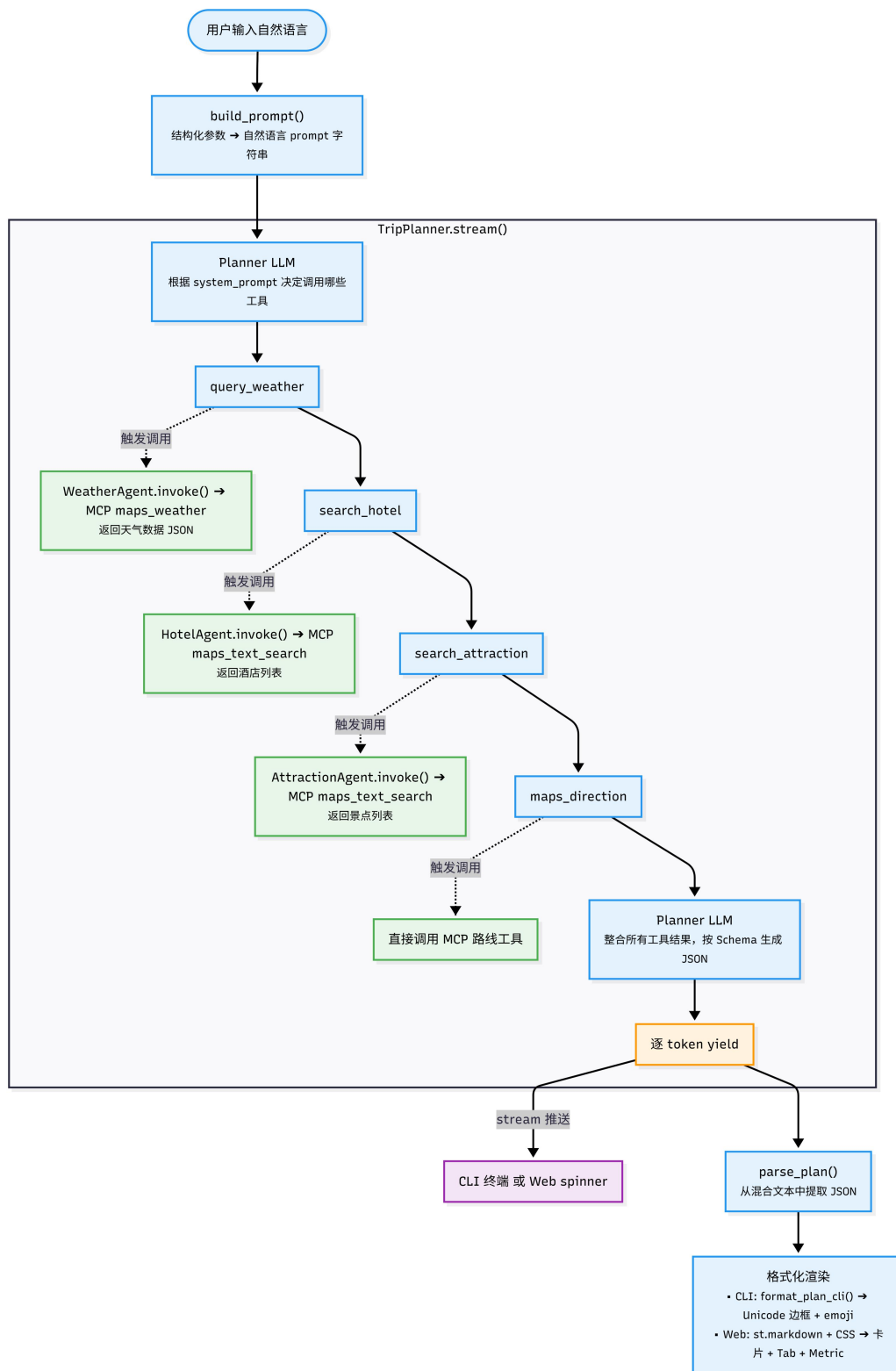


一次完整交互从 Web 表单开始。用户填写目的地、时间范围和偏好后，app.py 的 build_prompt 会把这些表单字段组织成一句更自然的旅行需求。随后 TripPlanner.stream 接收这段需求并启动 LangGraph Agent。由于系统使用流式事件，页面可以在 Agent 工作时看到“查询天气中”“搜索酒店中”“搜索景点中”“规划路线中”等状态，而不是让用户盯着空白页面等待。

在内部流程上，TripPlanner 首次运行会调用 build 方法。这个方法先通过 McpClientManager 获取三类工具：poi 工具用于景点和酒店搜索，weather 工具用于天气查询，route 工具用于路线规划。然后系统创建三个 SpecialistAgent，并把它们包装成 search_hotel、search_attraction、query_weather 三个 LangChain Tool。这样总控 Agent 看到的是统一工具接口，内部却可以由子 Agent 完成更细的领域推理。

这种设计避免了单个 Agent 既要理解全局行程、又要掌握所有工具细节的问题。总控 Agent 只关注“什么时候调用哪个能力”，子 Agent 只关注“如何完成单一领域查询”。从工程维护角度看，它比一个超长 Prompt 更容易调试，也更容易扩展。

3.3 数据流设计



如图所示，系统的数据流从用户自然语言输入开始。Web 端首先通过 `build_prompt()` 将结构化表单参数转换为自然语言 prompt，然后交给 `TripPlanner.stream()`。Planner LLM 根据 `system_prompt` 判断需要调用哪些工具，依次触发天气查询、酒店搜索、景点搜索和路线规划。天气、酒店、景点任务分别由对应的 Specialist Agent 调用 MCP 工具完成，路线规划则由 Planner 直接调用 MCP 路线工具。所有工具结果返回后，Planner LLM 按预设 Schema 整合为 JSON，并通过流式 token yield 返回给 CLI 或 Web 端。最后，`parse_plan()` 从混合文本中提取 JSON，再交给 CLI 或 Streamlit 页面进行格式化渲染。

四、关键实现与代码展示

4.1 Agent 核心循环

Agent 核心循环位于 `TripPlanner.stream`。这个函数不是简单等待最终答案，而是监听 LangGraph 的事件流。模型生成普通 token 时，系统把 token 继续传给前端；工具开始调用时，系统把工具名映射成中文状态提示；工具结束时则保持安静，避免页面被大量中间信息刷屏。

```
async for event in self._agent.astream_events(
    {"messages": [{"role": "user", "content": user_input}]},
    version="v2",
):
    kind = event.get("event", "")
    if kind == "on_chat_model_stream":
        content = event["data"]["chunk"].content
        content = _TOOL_CALL_PATTERN.sub("", content)
        if content.strip():
            yield content
    elif kind == "on_tool_start":
        name = event.get("name", "unknown")
        emoji, label = TOOL_LABELS.get(name, ("tool", name))
        yield f"\n{emoji} {label}...\n"
```

这段代码体现了 Agent 产品化时一个很实际的问题：用户不仅关心最终结果，也关心系统是不是正在工作。流式状态可以降低等待焦虑，同时也便于调试工具调用顺序。

4.2 子 Agent 包装为工具

本项目比较有代表性的实现，是把子 Agent 包装成 Tool。HotelAgent、AttractionAgent 和 WeatherAgent 内部本身也是 Agent，但对 TripPlanner 来说，它们只是三个可调用函数。这种“Agent as Tool”的写法让系统层次更清楚，也使总控 Agent 的提示词更容易管理。

```

@tool
async def search_hotel(query: str) -> str:
    return await self._hotel_agent.invoke(query)

@tool
async def search_attraction(query: str) -> str:
    return await self._attraction_agent.invoke(query)

@tool
async def query_weather(query: str) -> str:
    return await self._weather_agent.invoke(query)

all_tools = [search_hotel, search_attraction, query_weather, *route_tools]

```

这样做还有一个好处：权限边界更明确。天气 Agent 只拿天气工具，景点和酒店 Agent 只拿 POI 工具，路线工具则由 Planner 直接持有。即使模型在某个子任务中产生偏差，也不容易调用到不相关工具。

4.3 MCP 客户端与缓存

mcp_client.py 中的 McpClientManager 使用单例模式。这样设计主要是为了适配 Streamlit 的运行方式：Streamlit 每次用户交互都会重新执行脚本，如果每次都重新创建 MCP 客户端和工具列表，不仅耗时，也容易造成重复连接。单例和工具缓存让系统只在首次使用时加载工具，之后复用已有结果。

```

class McpClientManager:
    _instance: "McpClientManager | None" = None

    def __new__(cls) -> "McpClientManager":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

```

项目还加入了 RedisMcpCache，用于缓存天气和 POI 详情等相对稳定、重复率较高的 MCP 调用结果。缓存采用 fail-open 设计，也就是 Redis 不可用时只关闭缓存，不影响核心规划功能。运行日志中出现过“Redis MCP cache disabled: No module named 'redis'”，但系统仍然可以继续走 MCP 调用链，这说明缓存层没有绑死核心流程。

4.4 配置文件与上游兼容修复

config.py 不只是存放 API Key，它还统一管理模型名称、温度、MCP URL、传输协议、缓存开关和工具领域映射。项目中比较典型的工程处理是对 ChatTongyi 的流式 tool_calls 差分逻辑做了

Monkey-Patch。原因是上游库在处理流式工具调用增量时，可能直接访问 `prev_function['name']` 或 `prev_function['arguments']`，而某些 chunk 并不包含这些字段，最终触发 `KeyError`。

```
if "name" in function and "name" in prev_function:
    function["name"] = function["name"].replace(prev_function["name"], "")
if "arguments" in function and "arguments" in prev_function:
    function["arguments"] = function["arguments"].replace(
        prev_function["arguments"], ""
    )
```

这个修复虽然代码不长，但体现了 AI 应用开发中经常遇到的问题：很多故障并不在业务逻辑本身，而在模型 SDK、协议适配器、流式事件或外部服务兼容性上。能把这些问题定位并修掉，是项目能真正运行起来的重要部分。

4.5 Web 端实现

Web 端使用 Streamlit 实现。侧边栏负责收集旅行参数，主区域负责展示生成结果。为了避免每次点击都重新创建规划器，`app.py` 使用 `@st.cache_resource` 缓存 `TripPlanner` 实例；为了保留生成结果，使用 `st.session_state` 保存 `plan_data`、`plan_raw` 和 `status_lines`。

页面展示上，天气信息被渲染为卡片，每日行程放入 Tab，预算用 `st.metric` 呈现，用户还可以下载 Markdown。这样的设计虽然不复杂，但已经具备一个小型 AI 工具产品的基本体验：输入明确、状态可见、输出结构清晰、结果可以保存。

五、测试与评估

5.1 功能测试

项目目录中包含多种测试和诊断脚本，说明开发过程并不是只跑一次 Demo，而是围绕环境、协议、MCP 服务和配置做了多轮验证。`check_env.py` 用于检查 Python 版本、关键依赖、API Key 和网络连通性；`test_mcp.py` 用于验证 `McpClientManager` 能否成功获取高德地图工具；`test_adapter_usage.py` 则比较了不同 MCP 连接方式。

测试项	脚本/位置	验证目标
环境诊断	<code>check_env.py</code>	检查 Python、依赖、API Key、网络与 MCP 端点。
MCP 连接	<code>test_mcp.py</code>	验证是否能获取 MCP 工具列表。
传输协议	<code>test_adapter_usage.py</code> /	比较 <code>streamable-http</code> 、 <code>SSE</code> 、 <code>direct MCP</code> 等连接

	test_transport.py	方式。
配置验证	verify_config.py	检查 MCP_TRANSPORT、MCP_URL、缓存开关等配置。
Web 功能	app.py + 示例截图	验证表单输入、状态展示、结果渲染和下载。

5.2 Agent 行为评估

Agent 的评估不能只看回答有没有文字，还要看它是否按预期调用工具、是否生成可解析 JSON、是否覆盖旅行计划所需字段。对于本项目来说，天气、酒店、景点这类事实型信息必须走工具查询，不能让模型凭空回答；最终 JSON 必须能被 parse_plan 成功解析，否则 Web 页面就无法稳定展示。

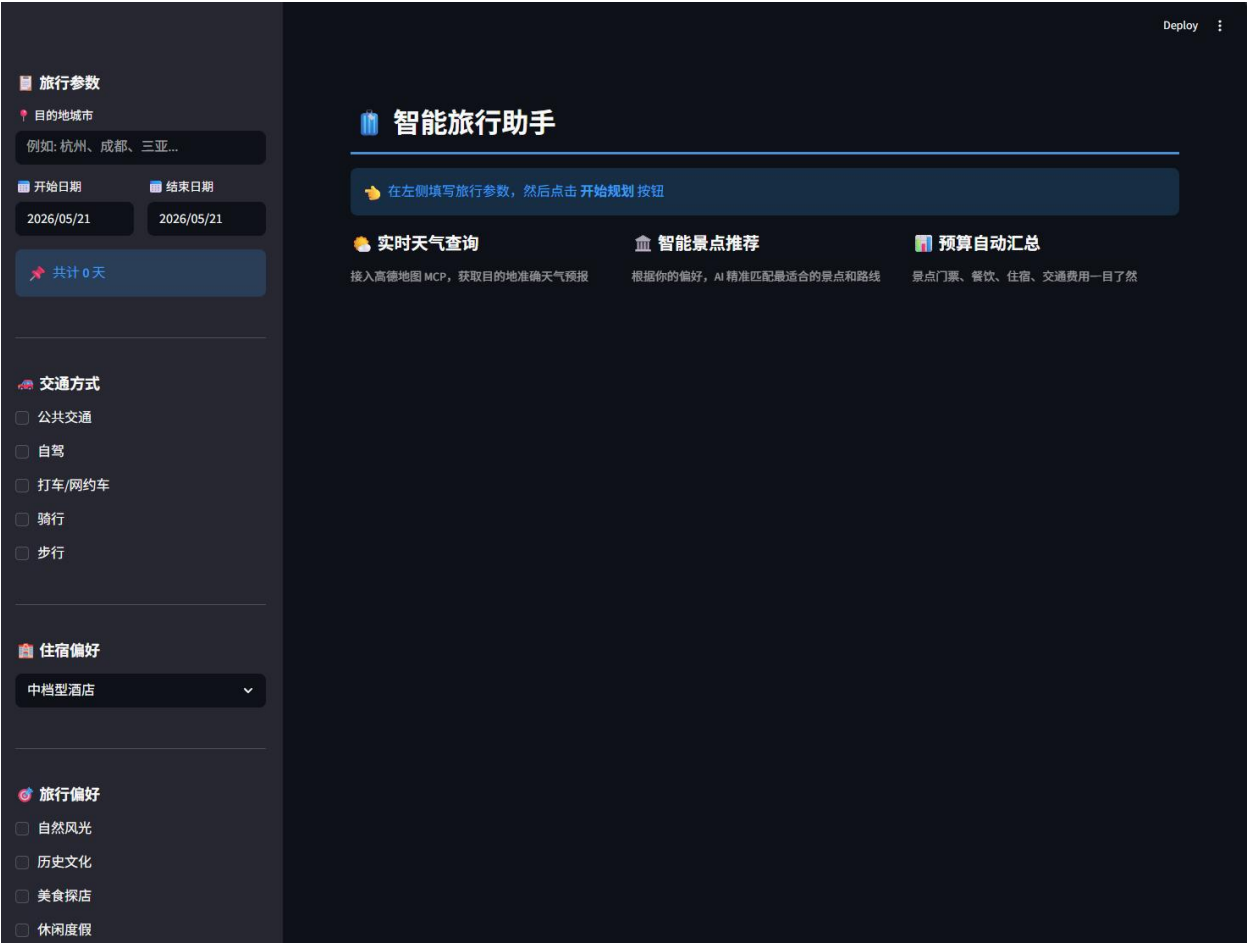
另外，行程本身也要评估合理性。每天是否安排 2 到 3 个景点，是否包含三餐，住宿是否和活动区域匹配，预算是否有分项和总计，天气信息是否覆盖每一天，这些都可以作为行为评估指标。相比普通问答任务，旅行规划更强调“可执行性”，所以评估时不能只看语言是否通顺，还要看安排是否能真正拿来使用。

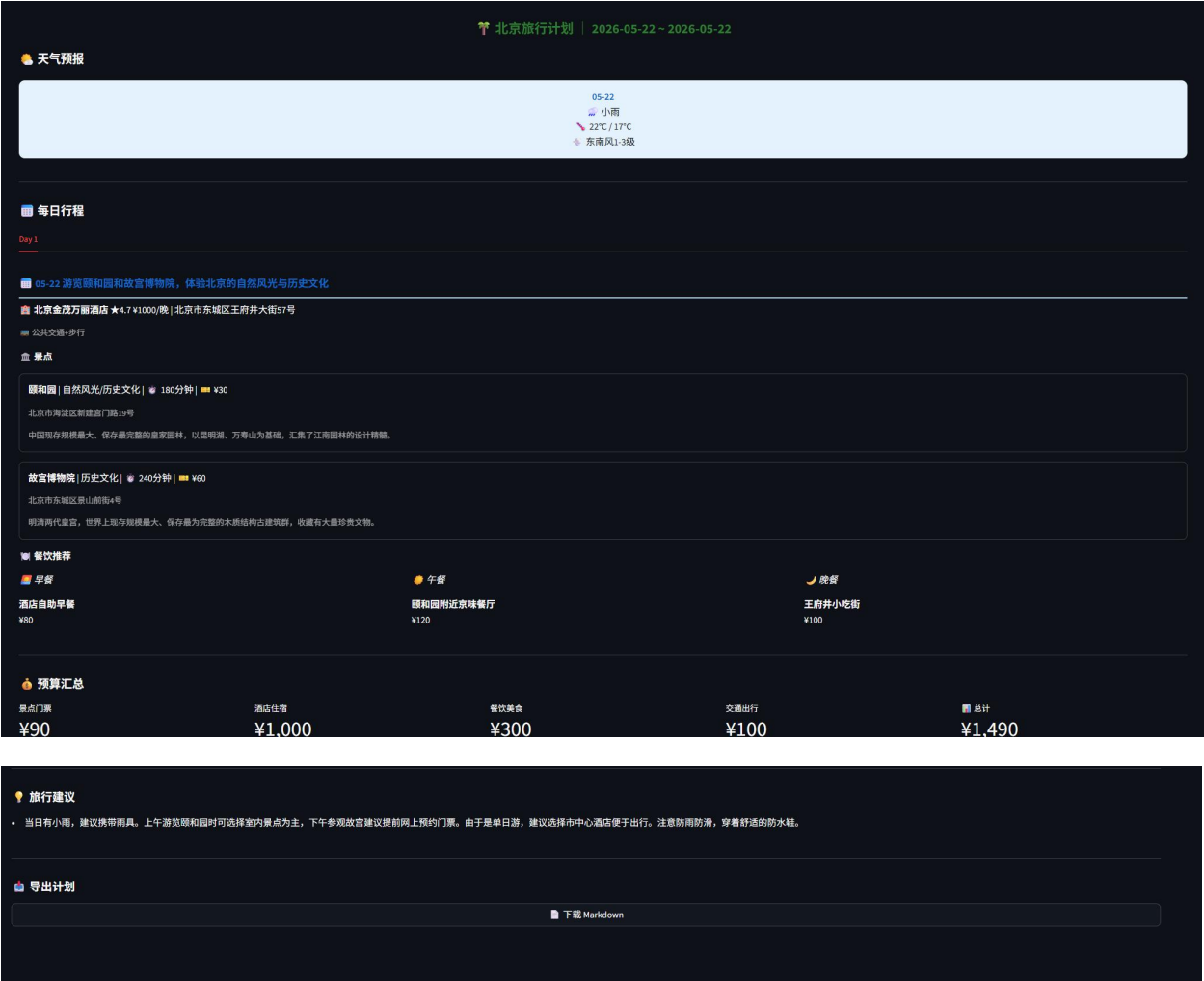
5.3 Benchmark 设计

为了后续持续改进，可以建立一组标准测试输入。例如选择长沙、杭州、成都、北京等不同城市，分别设置 1 日游、3 日游、5 日游，不同预算和不同偏好。每次修改提示词、模型或 MCP 连接方式后，都用同一组输入测试，观察 JSON 可解析率、工具调用覆盖率、字段完整率和总耗时。

评估维度	指标	含义
结构稳定性	JSON 可解析率	模型输出能否被 render.parse_plan 正常解析。
工具使用	事实型任务工具调用覆盖率	天气、酒店、景点是否实际调用 MCP 工具。
内容完整性	字段覆盖率	days、weather_info、hotel、attractions、meals、budget 是否齐全。
行程合理性	路线与时间冲突数	景点距离是否过远，游玩时间是否明显不合理。
性能表现	首 token 时间 / 总耗时	用户等待体验是否可接受。

5.4 Demo 截图





六、系统升级与扩展

6.1 可扩展架构

当前系统的扩展点比较清晰。新增能力时，可以新增一个 `SpecialistAgent`，也可以直接把新的 MCP 工具加入 `TripPlanner` 的工具列表。例如要增加餐饮推荐，可以创建 `FoodAgent`，并给它分配餐厅搜索、周边搜索和详情查询工具；要增加火车票或机票能力，可以创建 `TransportAgent`；要增加行程复盘和质量检查，可以创建 `EvaluatorAgent`。

这种架构比把所有逻辑写在一个函数里更适合长期迭代。每个 Agent 都有独立的提示词和工具集合，调试时可以单独观察某个领域的输出。`McpClientManager` 负责统一连接和缓存，也避免了每个模块都重复写连接逻辑。

6.2 下一阶段计划

下一阶段首先要解决稳定性问题。MCP 连接涉及外部服务、API Key、传输协议和依赖版本，任何一环不稳定都会影响 Demo，所以需要加入更明确的超时、重试和错误提示。其次要加强质量评估，建立标准测试集，避免每次只凭一次成功结果判断系统好坏。

产品层面可以继续增强交互能力，例如允许用户对某一天行程进行局部调整，而不是重新生成全部计划；允许用户锁定某个酒店或景点，让 Agent 只重排剩余部分；允许导出 PDF、日历或地图链接。部署层面则可以补充 Dockerfile、环境变量模板和云端部署说明，使项目更容易复现。

6.3 AI 能力演进路径

从能力演进看，本项目目前处于“工具增强型 Agent”阶段，核心目标是让模型能够查询真实工具并生成结构化结果。下一步可以进入“可评估 Agent”阶段，在生成后增加自检：比如检查每天景点数量是否合理、雨天是否安排过多户外项目、预算是否超过用户要求。再往后可以加入记忆模块，让系统记住用户喜欢自然风光、讨厌赶路、偏好市中心住宿等长期偏好。

更长期的方向是形成自治式旅行助理。它不仅能规划，还能接入预订、退改、实时天气提醒、交通延误提醒和临时改线。那时 Agent 的角色就不只是生成一份计划，而是陪伴用户完成整个出行过程。

七、课程总结

7.1 个人收获

在课堂上，我了解到了前沿的 agent 知识，认识到单纯地学好前后端已经不能适应行业的要求，要学会使用 AI，让 AI 帮助我完成简单繁琐的任务，更重要的是，要将 AI 能力深度整合到现有的业务场景与系统架构中。过去，我的视线可能过多地局限于如何利用 Spring Boot 搭建稳定的服务、如何通过 Redis 优化分布式缓存，或是死磕各类框架的底层源码。这些扎实的后端工程经验固然是不可或缺的基石，但在大模型飞速迭代的今天，仅仅停留在“实现业务逻辑”层面已经不足以构成绝对的核心竞争力。真正的技术壁垒，正在向“如何让系统具备思考与自主决策能力”转移。从单纯的“代码编写者”转型为“智能工作流的构建者”和“AI 驱动系统的架构师”，这不仅是技术栈的拓宽，更是思维方式的重塑。只有主动拥抱变化，将传统的工程化能力与前沿的深度学习、Agent 技术深度融合，才能在未来的行业变革中立于不败之地。

通过这个项目，我对 AI 应用开发的理解发生了比较明显的变化。最开始容易把大模型应用理解成“写好 Prompt，让模型回答”，但真正做旅行规划时会发现，Prompt 只是入口，系统能不能用，更多取决于工具接入、数据结构、异常处理和界面展示。一个看似简单的“帮我规划三日游”，背后其实包含任务拆解、事实查询、路线组织、预算计算和结果渲染。

项目中最有收获的部分是 Multi-Agent 的拆分。总控 Agent 不需要亲自处理所有细节，子 Agent 也不需要理解全局目标。它们通过工具接口连接起来，各自承担一部分职责。这种设计让我更直观地理解了 Agentic AI 和传统问答系统的区别：Agent 不是只生成文本，而是在一组工具和约束中完成任务。

7.2 工程思维转变

这个项目也让我意识到，AI 项目不是“模型效果好就结束”。实际开发中会遇到 API Key 配置、MCP 服务开通、依赖版本兼容、Streamlit rerun、Redis 缓存缺失、流式 tool_calls bug 等一系列工程问题。很多时候，真正花时间的不是写生成逻辑，而是把链路打通、把错误定位清楚、把结果稳定展示出来。

因此，我对工程化的理解从“实现功能”进一步变成了“让功能可运行、可诊断、可扩展”。报告中的测试脚本、配置管理和异常提示虽然不像界面那样直观，但它们决定了项目能不能在不同环境中复现。

7.3 对课程的建议

如果后续课程继续围绕 Agentic AI 展开，建议增加一些工具调用和 MCP 的实操内容。相比单纯讲 Prompt，工具调用更能体现大模型应用和真实系统之间的关系。也建议课程中加入标准化评估环节，例如要求每个项目提供一组固定测试输入，说明成功率、失败样例和改进方向。这样可以避免只展示一次成功 Demo，而忽略系统稳定性。